

Multi-Agent Systems: Final Homework Assignment RESIT

Ángel A. Gil (2788988)

MSc AI, VU - 03/03/2023

1 Monte Carlo simulation

The COVID scare was a wake-up call, highlighting the vulnerability of our hyper-connected modern world to the threat of pandemics. Now that the COVID urgency has passed, governments across the globe are rushing to update their emergency action plans and disaster scripts.

As part of this effort, you've been hired by a medical team that has been tasked with developing fast procedures to detect a blood-borne virus. Since these tests need to be administered to large groups in the population, and testing resources are limited, the medics have come up with the following procedure. They started from the assumption that they need to test N blood samples (of as many different individuals) and that N is large (e.g. $N = 10^6$). Furthermore, the probability that an individual is infected is p , where p is relatively small, e.g. $p < 0.1$. Based on these assumptions they propose the following procedure **to minimise the number of tests** they have to run: Rather than testing each sample individually, take a batch of k samples and mix them. Then this mixed sample is tested for the presence of the viral antigen:

- If the mixed sample tests **negative** (i.e. no viral antigen is detected), then all the individual samples were clear, and you therefore have the result for all k individual samples that went into the batch.
- If the mixed sample tests **positive** (i.e. the viral antigen is present indicating infection), one needs to retest all individual samples that went into the batch, in order to find out which individual(s) are actually infected.

Questions

1. Use Monte Carlo simulation to estimate the optimal batch size k (i.e. the one that minimises the expected number of tests) for a given value of p where p can take values between 10^{-1} and 10^{-4} .
2. Quantify the expected reduction in workload (compared to testing all samples individually).

Answers

Preamble: analytical solution

Before stating my claim, I want to emphasize that, although I am sure that the result is correct, the formalization might contain mistakes. I will use this result to compare those of the empirical MC simulation.

Claim Let T_k be the random variable associated to the number of tests of batch size k that will have to be performed. Then, the batch size that minimizes the expected number of tests $E[T_k]$ is

$$k^* = \sqrt{\frac{1}{p}}. \quad (1)$$

Proof We know for certain that we can decompose $T_k = \frac{N}{k} + N \cdot F$ where F is proportion of failed tests. The first term corresponds to the fixed number of batched tests to be performed in a first sweep, regardless of whether they are positive or negative. The second term is the additional number of (individual) tests that will have to be performed given the amount of batched test that turned out to be positive. Let us work the expectancy:

$$E[T_k] = E\left[\frac{N}{k} + N \cdot F\right] = \frac{N}{k} + N \cdot E[F] = \frac{N}{k} + N \cdot \min(1, k \cdot P)$$

$E[F] = N \cdot \min(1, k \cdot P)$ is not that straightforward: in short, we have $\frac{N}{k}$ batched tests, but each one of them consists of k individuals, hence the N . Then, since each individual has probability p to be infected, $k \cdot p$ is the expected number of infected individuals in each batch. Since we want proportion of failed tests, we have to limit the maximum of this expectancy to 1. For the sake of brevity, I will omit the 1 because it does not alter the final result, but it is important for implementation purposes - computing the expected total number of tests.

If we now minimize (i.e. derive with respect to k and equal to 0):

$$\frac{\partial E[T_k]}{\partial k} = -\frac{N}{k^2} + N \cdot P = 0 \iff N = N \cdot P \cdot k^2 \iff k = \sqrt{\frac{1}{P}}.$$

Since $\frac{\partial^2 E[T_k]}{\partial k^2} = \frac{2N}{k^3} > 0$ we can conclude that $k^* = \sqrt{\frac{1}{P}}$ is a minimum. $\square$

Question 1

For the sake of saving some computation time, and given that the optimal k does not depend on N , I chose a slightly smaller value $N = 10^5$. Let us start by performing MCs for fixed values of p in the range between 10^{-1} and 10^{-4} . Specifically, I will test $p = 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}$. From the claim, we know that the respective optimal batch sizes are: $k = \sqrt{10}, 10, 10\sqrt{10}, 100 \approx 3, 10, 32, 100$, but we will not use that for now.

- **Use case a)** $N = 10^5, p = 10^{-1}$.

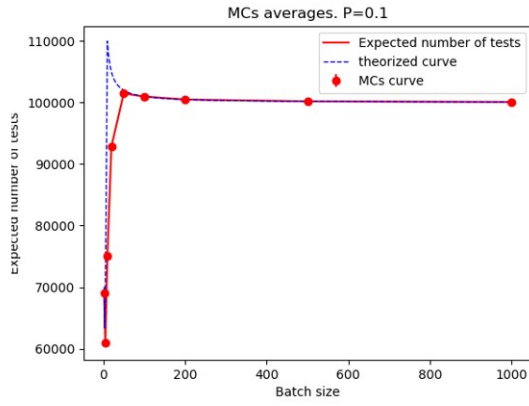
Instead of trying all possible values of k from 1 up to a high number, say 1000, I decided to perform a grid search, taking as pilot values $k = 2, 5, 10, 20, 50, 100, 200, 500, 1000$. For each of these values, I performed 5000 trials of MCs. Figure 1 shows a summary of the results. There seems to be an optimal between 2 and 10, so the search will be narrowed down to this interval.¹

For the second test, the considered interval is $k = [3, 9]$. Figure 2 shows the results of the test, where the optimal batch size is deduced to be 4 with, approximately, 59.396 expected number of tests with a standard deviation of 303, which is rather high considering how close $k=4$ and $k=3$ are. A t-test concluded that the difference in means is significant (p-value < 0.05), so we can assume that the optimal batch size for $p=0.1$ is 4.

- **Use cases b), c) and d)** $p = 10^{-2}, 10^{-3}, 10^{-4}$

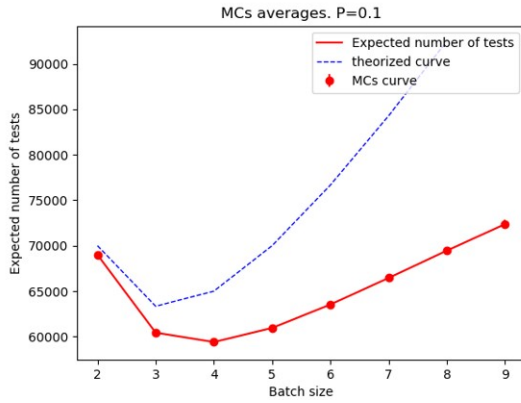
The same procedure was followed with the other 3 cases studied. I will omit the intermediate steps of the grid search and only present the first and final result of each instance. Figures 3 and 4 show the results when $p = 10^{-2}$. Although $k=11$ is the best result in the showcased picture, the standard deviations are high and it is not possible to decide for a specific value, but rather for the interval $[10, 12]$. Figures 5 and 6 show the results when $p = 10^{-3}$. Here, the averages come really close but the standard deviations do not, which makes impossible to choose an specific value again. The interval in this case is wider, ranging from 28 to 36. Same dynamics for the last case, $p = 10^{-4}$. Although 100 seems to be performing best, the standard deviations are so high that there would not a difference in the outcome even if we augmented the number of tests considerably.

¹As a side-note, it looks like the theorized result is slightly off, specially when $n \cdot k \approx 1$. Nevertheless, I will keep it in the report hoping that it results in an interesting exercise to the reader. Given the high sd, it is not possible to draw a conclusion without further analysis



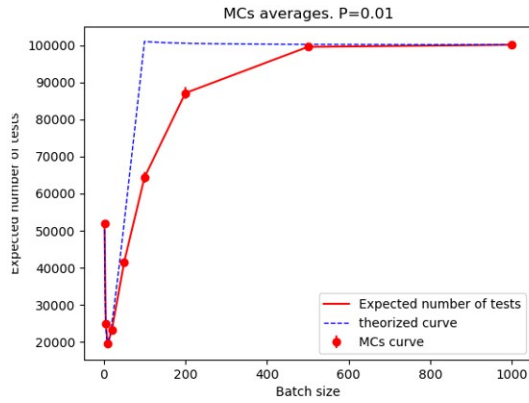
```
N=1e+05, p=1e-01
The expected number of tests for k=2 is 68998.36 (std=174.08)
The expected number of tests for k=5 is 60951.29 (std=351.28)
The expected number of tests for k=10 is 75134.28 (std=478.63)
The expected number of tests for k=20 is 92833.99 (std=456.95)
The expected number of tests for k=50 is 101484.46 (std=159.09)
The expected number of tests for k=100 is 100997.34 (std=16.21)
The expected number of tests for k=200 is 100500.00 (std=0.00)
The expected number of tests for k=500 is 100200.00 (std=0.00)
The expected number of tests for k=1000 is 100100.00 (std=0.00)
-----
p=1e-01, N=1e+05:
-----
From the batches considered, the optimal batch size
is 5, with 60951.29 expected number of tests and std of 351.28
```

Figure 1: Initial batches' results for $p = 0.1$



```
N=1e+05, p=1e-01
The expected number of tests for k=2 is 69000.30 (std=174.81)
The expected number of tests for k=3 is 60432.82 (std=244.84)
The expected number of tests for k=4 is 59397.97 (std=302.55)
The expected number of tests for k=5 is 60948.56 (std=347.90)
The expected number of tests for k=6 is 63527.01 (std=383.60)
The expected number of tests for k=7 is 66447.34 (std=417.57)
The expected number of tests for k=8 is 69454.66 (std=433.35)
The expected number of tests for k=9 is 72366.46 (std=457.96)
-----
p=1e-01, N=1e+05:
-----
From the batches considered, the optimal batch size
is 4, with 59397.97 expected number of tests and std of 302.55
```

Figure 2: Final batches' results for $p = 0.1$



```
N=1e+05, p=1e-02
The expected number of tests for k=2 is 51990.43 (std=62.31)
The expected number of tests for k=5 is 24900.06 (std=150.87)
The expected number of tests for k=10 is 19567.67 (std=297.01)
The expected number of tests for k=20 is 23205.90 (std=548.84)
The expected number of tests for k=50 is 41499.57 (std=1087.25)
The expected number of tests for k=100 is 64357.04 (std=1518.82)
The expected number of tests for k=200 is 87083.16 (std=1514.81)
The expected number of tests for k=500 is 99546.30 (std=571.77)
The expected number of tests for k=1000 is 100095.60 (std=69.14)
-----
p=1e-02, N=1e+05:
-----
From the batches considered, the optimal batch size
is 10, with 19567.67 expected number of tests and std of 297.01
```

Figure 3: Initial batches' results for $p = 0.01$

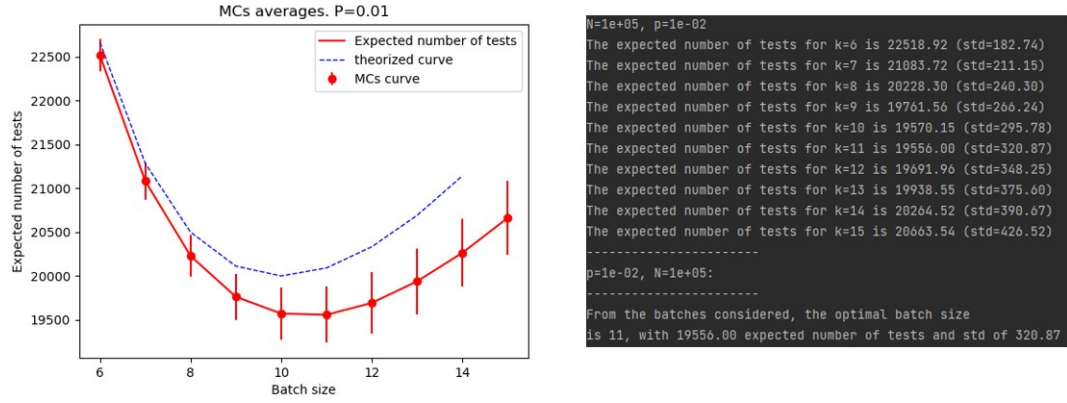


Figure 4: Final batches' results for $p = 0.01$

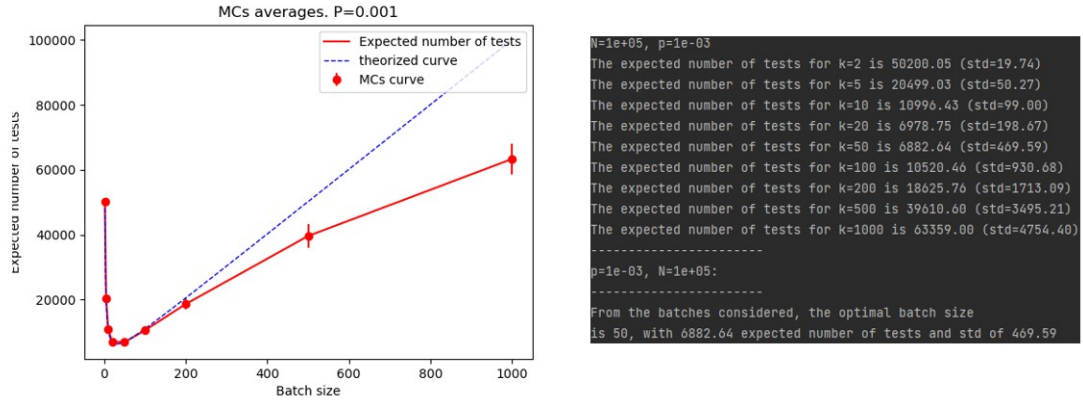


Figure 5: Initial batches' results for $p = 0.001$

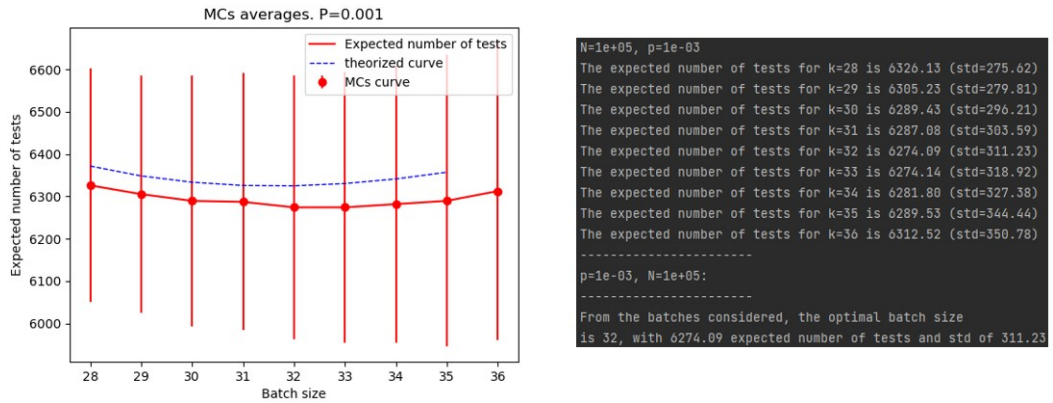


Figure 6: Final batches' results for $p = 0.001$

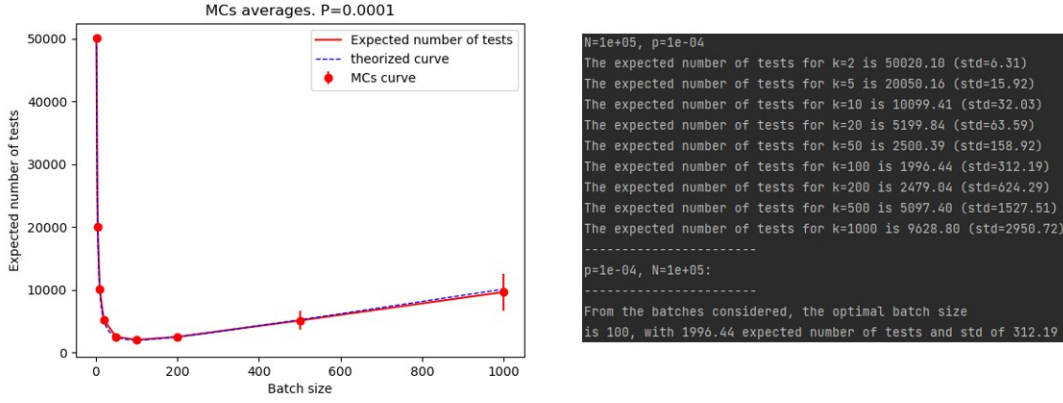


Figure 7: Initial batches' results for $p = 0.0001$

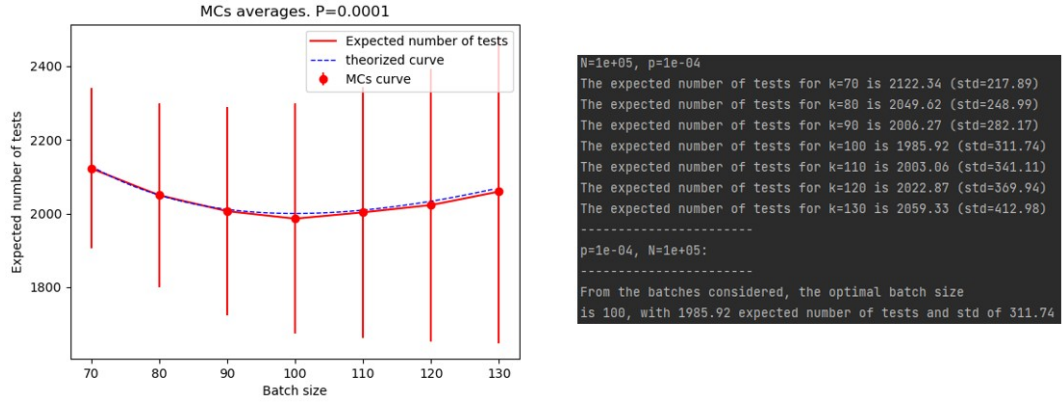


Figure 8: Final batches' results for $p = 0.0001$

Question 2

In order to quantify the expected reduction in workload, let's first visualize the value of k^* as p decreases and the average number of expected tests for each of these values. Figure 9 shows the optimal batch sizes as p increases and the expected average number of tests for each pair $(p, k^*(p))$. There is an inverse correlation between both the first and the second pair of variables.

By testing all samples individually, we would need 10^5 evaluations. Let's work out the 4 cases from question 1 and then make a final visualization.

- $p = 10^{-1}$. The obtained average number of tests (n) is 59.396. The expected reduction in workload is $(1 - \frac{59.396}{10^5}) * 100$, which yields a reduction of 40.6% in workload.
- $p = 10^{-2}$. $n=19.556 \rightarrow$ Reduction of 80.4%.
- $p = 10^{-3}$. $n=6.274 \rightarrow$ Reduction of 93.7%.
- $p = 10^{-4}$. $n=1.986 \rightarrow$ Reduction of 98.0%.

Figure 10 summarizes the expected reduction in workload as k^* increases i.e. as p decreases.

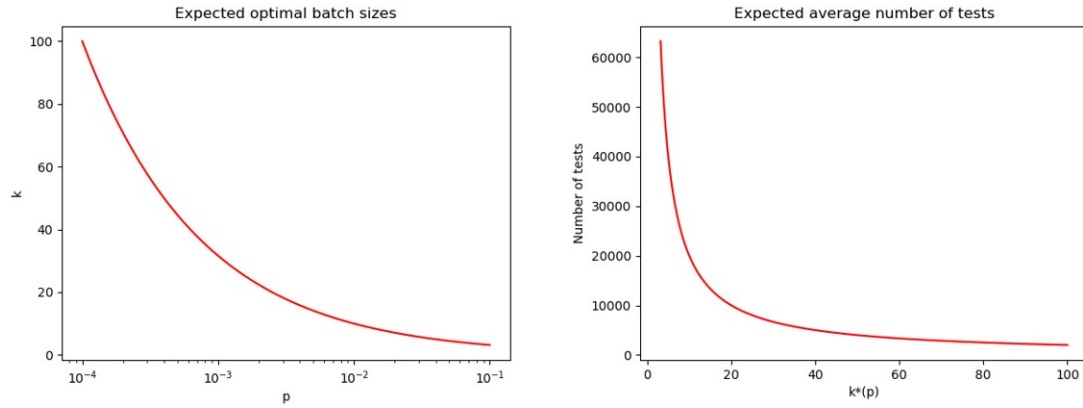


Figure 9: Left, optimal batch size (k^*) as p increases. Right, average number of tests for (p, k^*)

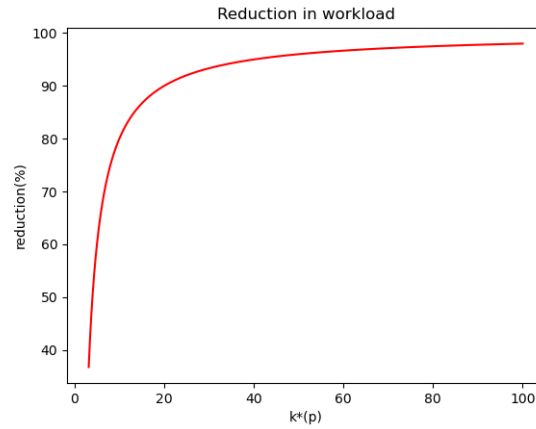


Figure 10: Reduction in workload

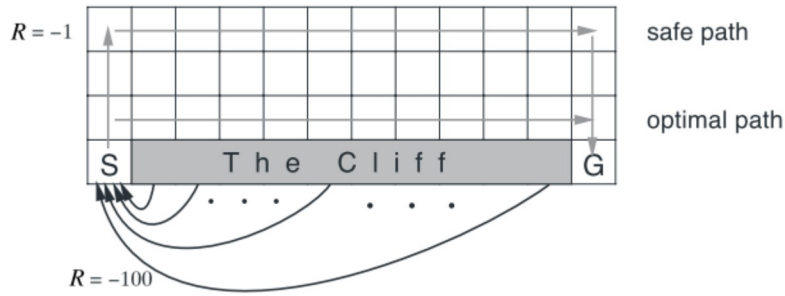
2 Reinforcement Learning: Cliff Walking

Consider the cliff-walking example (Sutton & Barto, ex. 6.6. p.108). Assume that the grid has 21 columns and 3 rows (above or in addition to the cliff). This is a standard undiscounted, episodic task, with start (S) and goal (G) states, and the usual actions causing movement up, down, right, and left. Reward is -1 on all transitions except:

- the transition to the terminal goal state (G) which has an associated reward of +20;
- transitions into the region marked *The Cliff*. Stepping into this region incurs a "reward" of -100 and also terminates the episode.

Questions

1. Use both SARSA and Q-Learning to construct an appropriate policy. Do you observe the difference between the SARSA and Q-learning policies mentioned in the text (safe versus optimal path)? For Q-learning, experiment with a replay-buffer. Discuss.
2. Try different values for ϵ (parameter for ϵ -greedy policy). How does the value of ϵ influence the result? Discuss.



3. Now assume that there is a snake pit in the cell on the top row and 11th column, i.e. right in the middle of the "safe path". Stumbling in this snake pit also carries a penalty (negative reward) of -100. Does this change the results obtained by SARSA or Q-learning? Discuss.

Answers

Question 1

Figures 11, 12 and 13 show the paths traversed using the policies obtained from the three algorithms in this order: SARSA, Q-Learning and Q-Learning with the buffer (Q-buffer). In the pictures, O indicates the path followed by the policy, '.' the unvisited regions and X, the cliff.



Figure 11: Optimal SARSA Policy



Figure 12: Optimal Q-Learning Policy

There is a clear difference between the policies: Q-learning goes Off-Policy, which takes higher risk of falling off the cliff but reaches the optimal policy. On the other hand, SARSA updates its policy relative to the current state-action pair, remaining On-Policy, which results in a safer path where the cliff edge is avoided but the goal is reached more slowly. Additionally, in order to obtain the desired policy, SARSA had to be trained for double the episodes than Q-learning (2000 for the former and 1000 for the latter). The rest of the settings remained equal, limiting the steps to 100, $\alpha = 0.1$, $\gamma = 0.9$ & $\epsilon = 0.1$. Figure 14 shows the heat-map for each policy.



Figure 15: Final policy, $\epsilon = 0.1$

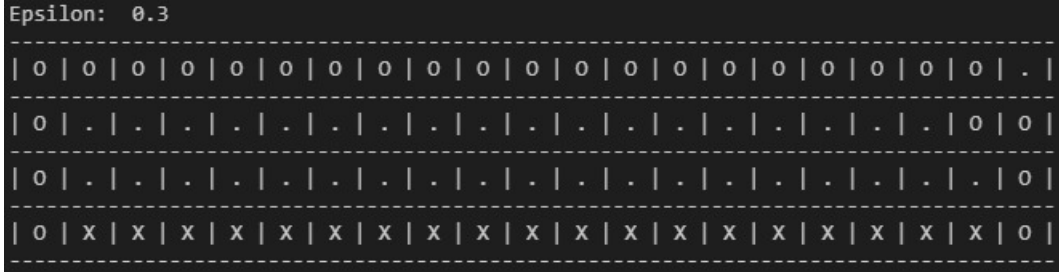


Figure 16: Final policy, $\epsilon = 0.3$

illustrates this issue by showing the rewards obtained batched by every 100 episodes for each algorithm on different epsilons. Lower ϵ -values lead to faster convergence on all algorithms, but the effect intensifies with Q-learning, where the convergence is much faster. Additionally, due to the replay, Q-buffer remains capable of constructing optimal policies at higher epsilon. It is observed that by reducing ϵ , the methods converge to the optimal policy. Q-learning with replay buffer converges in lesser number of episodes as compared to Q-learning. Even in this relatively simple environment, it highlights the enhanced stability provided by the replay buffer. However, this requires additional memory and additional computational resource.

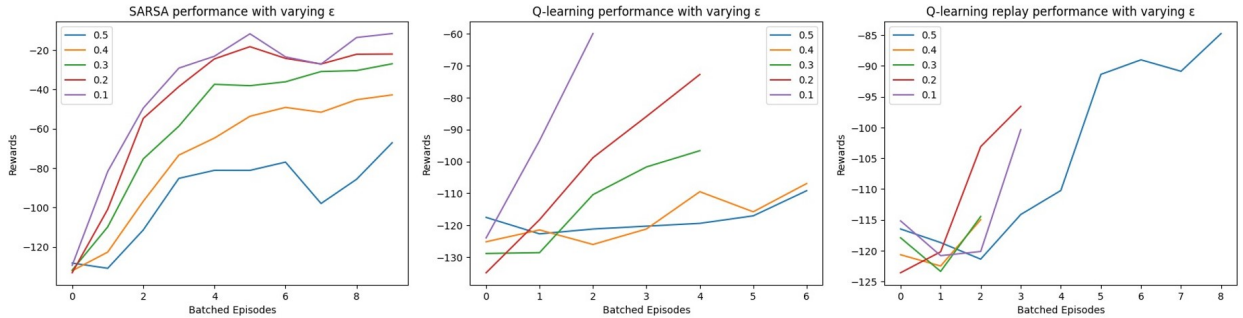


Figure 17: Algorithms' performance with different ϵ

Question 3

Figures 17 and 18 show the results of the addition of the snake-pit. The resulting policy obtained by SARSA changed, but there was no difference to the policy obtained by Q-learning with and without replay. This is not surprising because the snake-pit was not in the way of the optimal policy.

With the settings specified in the first question, SARSA was not able to converge, so it had to be trained

for 2000 episodes with a lower ϵ of 0.05. Lowering ϵ led SARSA to opt for a riskier policy and resulted in a path that goes around the snake-pit. In order to maintain a safe path to the goal, SARSA still moves away from the cliff after overcoming the snake-pit.

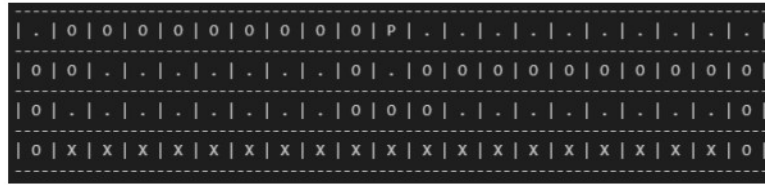


Figure 18: Optimal SARSA with Snake-pit



Figure 19: Optimal Q-Learning with Snake-pit